SERCOP — Servicio Nacional de Contratacion Publica
Coordinacion de Tecnologia de la Informacion

PRUEBA DE ESCRITORIO
SercoBot — Asistente Virtual de Contratacion Publica

Documento: PED-DIO-2026-206
Version: 1.0
Fecha: 16 de abril de 2026

Campo	Detalle
Documento	PED-DIO-2026-206
Sistema	SercoBot — Asistente Virtual de Contratacion Publica
Tipo de prueba	Prueba de escritorio (desk check) — trazabilidad estatica del codigo
Version sistema	2.0.0
Elaboro	Mauricio Ruiz — Analista de Sistemas TIC
Reviso	Paul Vasquez Mendez — Director de TIC
Aprobo	Pendiente
Fecha	16 de abril de 2026
Clasificacion	Uso interno SERCOP

1. Objetivo

Verificar estaticamente — sin ejecutar el sistema — que la logica de negocio de SercoBot produce los resultados esperados ante los principales escenarios de uso. Esta prueba de escritorio traza el flujo completo de cada escenario a traves de los modulos involucrados, identifica las decisiones de ramificacion y confirma la salida esperada.

2. Alcance

El presente documento cubre los siguientes modulos del sistema SercoBot:

Modulo	Archivo	Responsabilidad
Webhook handler	agent/main.py	Recibe mensajes de WhatsApp, deduplication, background tasks
Brain / orquestador	agent/brain.py	Shortcuts, tool calling, RAG, llamada al LLM
Tools	agent/tools.py	obtener_montos_pie, recomendar_tipo_contratacion, obtener_plazos, info_rup, obtener_fecha_hora_ecuador
Retriever RAG	agent/retriever.py	Busqueda hibrida pgvector (HNSW + tsvector) y reranking
Providers	agent/providers/	Normalizacion de mensajes WhatsApp (Meta Cloud API)
Memory	agent/memory.py	Historial de conversacion por telefono (PostgreSQL)

3. Definiciones

Termino	Definicion
Prueba de escritorio	Tecnica de verificacion estatica donde el analista traza manualmente la ejecucion del codigo linea por linea sin correrlo
RAG	Retrieval-Augmented Generation: recuperar fragmentos de normativa relevante antes de llamar al LLM
Tool calling	Mecanismo por el que el LLM decide invocar una funcion externa (tool) en lugar de responder directamente
Shortcut	Respuesta pre-cacheada que no pasa por el LLM — 0 tokens, <1ms
RRF	Reciprocal Rank Fusion: algoritmo que combina rankings de busqueda semantica y lexica
Reranker	Modelo cross-encoder que re-ordena los chunks recuperados por relevancia real
LLM	Large Language Model — en produccion: gemma4:e2b via Ollama local
PIE	Presupuesto Inicial del Estado — base para calcular umbrales de contratacion
Chunk	Fragmento de documento normativo indexado en pgvector (max 512 tokens)

4. Flujo general del sistema

Todo mensaje entrante sigue este recorrido antes de bifurcarse segun el tipo de contenido:

```
WhatsApp (usuario envia mensaje)
  -> Meta Cloud API -> POST /webhook (agent/main.py)
    -> proveedor.parsear_webhook()  [providers/meta.py]
      -> MensajeEntrante(telefono, texto, mensaje_id, es_propio)
        -> [es_propio?] -> ignorar
        -> [sin texto (multimedia)?] -> _responder_multimedia() -> enviar msg texto
        -> [mensaje_id en _mensajes_procesados?] -> ignorar (deduplicacion)
        -> background_tasks.add_task(_procesar_mensaje)
          -> [saludo corto?] -> NO enviar acuse
          -> [pregunta?]    -> enviar acuse '...Consultando normativa...'
          -> obtener_historial(telefono, limite=4)
          -> generar_respuesta(texto, historial, telefono)  [brain.py]
            -> [shortcut match?] -> retornar respuesta fija
            -> [tool needed?]   -> llamar tool(s) -> contexto enriquecido
            -> [RAG needed?]    -> buscar_contexto() -> reranker -> top-4 chunks
            -> LLM (Ollama gemma4:e2b) -> respuesta
          -> guardar_mensaje(user) + guardar_mensaje(assistant)
          -> proveedor.enviar_mensaje(telefono, respuesta)
```

5. Escenarios de prueba de escritorio

Se definen 9 escenarios que cubren los flujos criticos del sistema.

Escenario E-01	Saludo puro -> menu de bienvenida (shortcut)	Modulo: brain.py / shortcuts

Precondicion: el usuario envia unicamente un saludo sin pregunta.

Entrada: "hola"

Paso	Modulo / Funcion	Accion	Resultado parcial
1	main.py / _procesar_mensaje	_es_saludo = True (len('hola'.split()) <= 3 y sin keywords)	NO se envia acuse previo
2	main.py	obtener_historial(telefono, limite=4)	historial=[] (usuario nuevo)
3	brain.py / generar_respuesta	clasificar_mensaje('hola') -> tipo='saludo'	tipo='saludo'
4	brain.py / shortcuts	_cargar_faq_cache() busca patron 'hola' en faq_cache.yaml	match shortcut 'msg_bienvenida'
5	brain.py	retorna cfg['msg_bienvenida'] directamente sin llamar al LLM	respuesta = menu de bienvenida (5 categorias)
6	main.py	guardar_mensaje(telefono, 'user', 'hola')	persistido en BD
7	main.py	guardar_mensaje(telefono, 'assistant', respuesta)	persistido en BD
8	providers/meta.py	enviar_mensaje(telefono, menu_bienvenida)	POST /messages a Meta API -> 200 OK

Resultado esperado: El usuario recibe el menu de bienvenida con las 5 categorias (procesos, RUP, normativa, portal, garantias). No se llama al LLM. Tiempo de respuesta < 500ms (shortcut sin RAG).

Escenario E-02	Pregunta normativa -> RAG -> respuesta citando articulo	Modulo: brain.py / retriever.py

Precondicion: base de conocimiento activa con 3,059 chunks indexados.

Entrada: "cuales son las causales de suspension del RUP"

Paso	Modulo / Funcion	Accion	Resultado parcial
1	main.py	_es_saludo=False (len>3 y contiene keywords)	se envia acuse '...Consultando normativa...'
2	main.py	obtener_historial(telefono, limite=4)	historial reciente (max 4 mensajes)
3	brain.py	clasificar_mensaje() -> no es shortcut, no requiere tool de montos/plazos	flujo -> RAG
4	brain.py / LLM (1ra llamada)	LLM evalua si necesita tool: RUP puede beneficiarse de info_rup()	LLM decide llamar info_rup()
5	tools.py / info_rup	retorna dict con requisitos, costo, plazos y causales de suspension RUP	contexto enriquecido con datos estructurados
6	retriever.py / buscar_contexto	expand_query('causales suspension RUP') -> agrega sinonimos SERCOP	query expandida
7	retriever.py	embed query con nomic-embed-text (768d) via Ollama	vector de 768 dimensiones
8	retriever.py	pgvector HNSW coseno: top-12 chunks por similitud semantica	12 candidatos semanticos

9	retriever.py	GIN tsvector to_tsvector('spanish', texto) busqueda lexica	candidatos lexicos adicionales
10	retriever.py	RRF: fusiona rankings semantico + lexico -> top-12 unicos	lista RRF consolidada
11	retriever.py	cross-encoder reranker: puntua pares (query, chunk) -> top-4	top-4 chunks mas relevantes
12	retriever.py / formatear_contexto	formatea: '[FUENTE X: Art. Y LOSNCP]\n<texto>'	contexto ~1,200 tokens
13	brain.py	system_prompt += contexto RAG + resultado info_rup()	prompt enriquecido
14	brain.py / LLM (respuesta)	Ollama gemma4:e2b genera respuesta citando articulos del contexto	respuesta con 'Art. X LOSNCP'
15	main.py	guardar + enviar respuesta	usuario recibe respuesta con citacion legal

Resultado esperado: El usuario recibe una respuesta que lista las causales de suspension del RUP con citacion del articulo exacto (ej. Art. 16-18 LOSNCP). Si el contexto RAG no contiene el articulo, el LLM usa la frase anti-alucinacion: 'La normativa establece que [...], te recomiendo verificar el articulo en compraspublicas.gob.ec'.

Escenario E-03	Pregunta de monto -> tool obtener_montos_pie	Modulo: brain.py / tools.py

Precondicion: ninguna (tool no depende de RAG ni de la BD).

Entrada: "cuanto es el monto maximo de infima cuantia en 2025"

Paso	Modulo / Funcion	Accion	Resultado parcial
1	main.py	_es_saludo=False -> acuse previo enviado	acuse enviado
2	brain.py	prompt al LLM incluye definicion de tool obtener_montos_pie	LLM recibe schema de tools
3	brain.py / LLM (1ra llamada)	LLM detecta keywords 'monto', 'infima cuantia' -> decide tool call	tool_call: obtener_montos_pie()
4	brain.py / _ejecutar_tool	tools.py::obtener_montos_pie() -> dict con umbrales 2025 por tipo	{'infima_cuantia': '$7.263', 'menor_cuantia_bs': '$72.630', ...}
5	brain.py	resultado tool inyectado en historial como mensaje 'tool'	contexto enriquecido con valores exactos
6	brain.py / LLM (2da llamada)	LLM genera respuesta usando datos de la tool (NO el RAG)	respuesta con '$7.263' y base legal Art. 52.1 LOSNCP
7	main.py	guardar + enviar	usuario recibe monto exacto actualizado 2025

Resultado esperado: El usuario recibe el monto exacto de infima cuantia ($7,263 aprox. en 2025) con la base legal correcta (Art. 52.1 LOSNCP y Art. 195 RGLOSNCP). El RAG NO se consulta para preguntas de monto — los valores en el RAG pueden estar desactualizados respecto al PIE vigente.

Escenario E-04	Pregunta de plazo de proceso -> tool obtener_plazos	Modulo: brain.py / tools.py

Precondicion: ninguna.

Entrada: "cuantos dias tengo para impugnar una adjudicacion"

Paso	Modulo / Funcion	Accion	Resultado parcial

1	main.py	_es_saludo=False -> acuse previo	acuse enviado
2	brain.py / LLM (1ra llamada)	keywords 'impugnar', 'adjudicacion' -> tool obtener_plazos detectada	tool_call: obtener_plazos('impugnacion')
3	tools.py / obtener_plazos	tipo='impugnacion' -> retorna dict con plazo 3 dias habiles + Art. 102	{'plazo': '3 dias habiles', 'base_legal': 'Art. 102 LOSNCP'}
4	brain.py	resultado inyectado en historial como mensaje tool	contexto con plazos exactos
5	brain.py / LLM (2da llamada)	LLM redacta respuesta con datos de la tool	respuesta: '3 dias habiles desde notificacion de adjudicacion'
6	main.py	guardar + enviar	usuario recibe plazo exacto con base legal

Resultado esperado: El usuario recibe el plazo de impugnacion (3 dias habiles desde la notificacion de adjudicacion) con la base legal correcta. La tool obtener_plazos soporta 7 tipos: impugnacion, contrato, garantias, subasta_inversa, menor_cuantia, cotizacion, licitacion.

Escenario E-05	Comparacion de dos procesos -> tools en paralelo	Modulo: brain.py / tools.py

Precondicion: ninguna.

Entrada: "cual es la diferencia entre cotizacion y menor cuantia"

Paso	Modulo / Funcion	Accion	Resultado parcial
1	main.py	_es_saludo=False -> acuse previo	acuse enviado
2	brain.py / LLM (1ra llamada)	keywords 'diferencia', 'cotizacion', 'menor cuantia' -> tool calls multiples	tool_calls: [obtener_montos_pie(), recomendar_tipo_contratacion('cotizacion'), recomendar_tipo_contratacion('menor cuantia')]
3	tools.py / obtener_montos_pie	retorna umbrales 2025 de todos los tipos de proceso	{'cotizacion': {'min':'$72.630','max':'$544.725'}, 'menor_cuantia_bs':{'max':'$72.630'},...}
4	tools.py / recomendar_tipo_contratacion	recomendar('cotizacion') -> nombre, normativa, ventajas	contexto de cotizacion
5	tools.py / recomendar_tipo_contratacion	recomendar('menor cuantia') -> nombre, normativa, ventajas	contexto de menor cuantia
6	brain.py	los 3 resultados de tools se inyectan en historial	contexto consolidado con montos + descripcion de ambos procesos
7	brain.py / LLM (2da llamada)	LLM genera tabla comparativa con montos, plazos y normativa de ambos procesos	respuesta comparativa estructurada con emojis de proceso
8	main.py	guardar + enviar	usuario recibe comparacion clara con umbrales exactos en USD

Resultado esperado: El usuario recibe una comparacion estructurada: Menor Cuantia (hasta $72,630) vs Cotizacion ($72,630 - $544,725), con plazos de publicacion y cantidad de oferentes para cada uno. La respuesta incluye la regla practica: 'si supera $72,630 debes ir a cotizacion'.

Escenario E-06	Mensaje multimedia (audio/imagen) -> respuesta orientativa sin LLM	Modulo: main.py / providers

Precondicion: el usuario envia una nota de voz o imagen.

Entrada: mensaje tipo 'audio' (sin campo texto)

Paso	Modulo / Funcion	Accion	Resultado parcial
1	providers/meta.py / parsear_webhook	tipo='audio' -> MensajeEntrante(texto=None)	msg.texto = None
2	main.py / webhook_handler	if not msg.texto: background_tasks.add_task(_responder_multimedia)	LLM NO invocado
3	main.py / _responder_multimedia	proveedor.enviar_mensaje(telefono, msg_multimedia)	mensaje orientativo: 'Solo puedo responder preguntas de texto'
4	main.py	NO se guarda en historial (no es conversacion valida)	BD sin cambios

Resultado esperado: El usuario recibe en < 1 segundo un mensaje orientativo indicando que el bot solo procesa texto, con ejemplos de preguntas validas. No se consume ninguna llamada al LLM ni al RAG.

Escenario E-07	Mensaje duplicado por reintento de Meta -> deduplicacion	Modulo: main.py

Precondicion: Meta reintenta el webhook porque no recibio 200 OK en 5s (LLM tardo mas).

Entrada: segundo POST del webhook con el mismo mensaje_id que ya fue procesado

Paso	Modulo / Funcion	Accion	Resultado parcial
1	main.py / webhook_handler	parsear_webhook() -> msg.mensaje_id = 'wamid.XXXX'	mensaje_id obtenido
2	main.py	if msg.mensaje_id in _mensajes_procesados: -> True	duplicado detectado
3	main.py	logger.info('Mensaje duplicado ignorado: wamid.XXXX')	log de auditoria
4	main.py	continue -> no se agrega background_task	LLM NO invocado, BD sin cambios
5	main.py	retorna {'status': 'ok'} a Meta de inmediato	Meta no reintenta mas
6	main.py (mantenimiento)	if len(_mensajes_procesados) > 500: purgar primeros 250	cache limitado a 500 IDs max

Resultado esperado: El mensaje duplicado es ignorado silenciosamente. El usuario NO recibe una segunda respuesta identica. La cache de IDs se mantiene en maximo 500 entradas para evitar crecimiento ilimitado.

Escenario E-08	Error en LLM / timeout de Ollama -> notificacion graceful al usuario	Modulo: main.py / brain.py

Precondicion: Ollama no responde (caido o timeout > 120s).

Entrada: cualquier pregunta valida, Ollama retorna ConnectionError

Paso	Modulo / Funcion	Accion	Resultado parcial
1	brain.py / _llamar_ollama	httpx.AsyncClient.post() timeout=120s -> TimeoutException	excepcion capturada
2	brain.py	except Exception as e: logger.error(f'Error Ollama: {e}')	log del error
3	brain.py	raise -> excepcion sube a _procesar_mensaje	generar_respuesta() lanza excepcion
4	main.py / _procesar_mensaje	except Exception as e: logger.error + traceback.format_exc()	traza completa en logs (journalctl)
5	main.py	proveedor.enviar_mensaje(telefono,	usuario recibe: 'Tuve un problema

		mensaje_error_usuario)	tecnico... intenta de nuevo o llama al 1800-737267'
6	main.py	BD: NO se guarda mensaje de error en historial	conversacion limpia para reintento

Resultado esperado: El usuario recibe un mensaje de disculpa con orientacion alternativa (web + telefono) en lugar de silencio. El administrador puede diagnosticar via: journalctl -u sercobot -f | grep ERROR. La traza completa del stack aparece en los logs del servicio.

Escenario E-09	Pregunta fuera del scope de contratacion publica	Modulo: brain.py / LLM

Precondicion: ninguna.

Entrada: "cual es la capital de Francia"

Paso	Modulo / Funcion	Accion	Resultado parcial
1	main.py	_es_saludo=False -> acuse previo	acuse enviado
2	brain.py	no es shortcut, no activa ninguna tool	flujo -> RAG
3	retriever.py	buscar_contexto('capital Francia') -> similitud muy baja con chunks SERCOP	top-4 chunks no relevantes o vacios
4	brain.py	system_prompt incluye 'Solo responde sobre contratacion publica ecuatoriana'	constraint en prompt
5	brain.py / LLM	LLM no tiene contexto relevante + constraint en system_prompt	LLM rechaza la pregunta out-of-scope
6	brain.py	respuesta: 'Solo puedo ayudarte con temas de contratacion publica...'	msg_fuera_scope
7	main.py	guardar + enviar	usuario recibe orientacion al scope correcto

Resultado esperado: El usuario recibe un mensaje que aclara el scope del bot y ofrece retomar con preguntas de contratacion publica. El bot NO inventa respuestas sobre temas ajenos al SERCOP.

6. Tabla resumen de escenarios

N.	Escenario	Modulo principal	Tool / RAG	LLM invocado	Estado esperado
E-01	Saludo puro -> menu	brain.py	Shortcut	NO	Pasa
E-02	Pregunta normativa -> RAG + articulo	retriever.py	RAG + info_rup	SI	Pasa
E-03	Monto infima cuantia	tools.py	obtener_montos_pie	SI (x2)	Pasa
E-04	Plazo de impugnacion	tools.py	obtener_plazos	SI (x2)	Pasa
E-05	Diferencia cotizacion vs menor cuantia	tools.py	3 tools paralelas	SI (x2)	Pasa
E-06	Mensaje multimedia (audio/imagen)	main.py	Ninguna	NO	Pasa
E-07	Mensaje duplicado (reintento Meta)	main.py	Ninguna	NO	Pasa
E-08	Error en LLM / timeout Ollama	main.py	Ninguna	Falla gracefully	Pasa
E-09	Pregunta fuera de scope	brain.py	RAG (sin match)	SI	Pasa

7. Criterios de aceptacion

Criterio	Descripcion	Umbral
Precision normativa	Respuestas con articulo citado cuando el chunk lo incluye	>= 90%
Anti-alucinacion	Nunca se inventa un numero de articulo no presente en el chunk	100%
Montos correctos	Valores en USD siempre provienen de obtener_montos_pie (no del RAG)	100%
Tiempo de respuesta	Shortcut < 500ms | Tool < 10s | RAG + LLM < 30s	95% de mensajes
Deduplicacion	Ningun mensaje_id procesado mas de una vez	100%
Manejo de errores	Ante cualquier excepcion el usuario recibe mensaje de error (nunca silencio)	100%
Out-of-scope	Preguntas no relacionadas a contratacion publica redirigidas correctamente	100%
Multimedia	Mensajes no-texto atendidos sin invocar LLM	100%

8. Observaciones y riesgos identificados

Riesgo	Impacto	Probabilidad	Mitigacion
Citrix SSL DPI bloquea graph.facebook.com	Bot procesa y genera respuesta pero NO puede enviarla al usuario via Meta API	Alta (confirmado)	Solicitar a Franklin Arias agregar graph.facebook.com a lista de bypass SSL del firewall perimetral
git pull bloqueado en servidor	Despliegue de nuevas versiones requiere SCP manual	Alta (confirmado)	Solicitar bypass SSL para github.com o usar acceso por SSH key con token personal
Timeout Ollama (gemma4:e2b)	Respuestas lentas o caidas en picos de uso concurrente	Media	Monitorear con journalctl; escalar a gemma4:26b cuando se migre al servidor RHEL con mas RAM
Chunks RAG desactualizados para montos	Valores en USD de años anteriores devueltos por RAG	Baja (mitigado)	Sistema usa tool obtener_montos_pie para cualquier pregunta de monto — RAG no se consulta para estos casos
META_PHONE_NUMBER_ID incorrecto	Error 400 de Meta API al enviar mensajes	Media	Verificar el ID en Meta for Developers > WhatsApp > API Setup; regenerar token si expiro
Crecimiento ilimitado de _mensajes_procesados	Consumo de memoria RAM del proceso uvicorn	Baja (mitigado)	Cache limitado a 500 IDs con purga automatica al superar el limite